SPRING MVC

1) MVC Pattern

What is MVC?

MVC stands for:

M – Model
V – View
C – Controller

It is a design pattern used to separate application concerns.

Purpose of MVC

To separate:

• Business logic
• UI logic
• Data handling

This improves:

• Maintainability
• Scalability
• Testability
• Code readability

Components of MVC

1) Model

Represents:

- Application data
- Business logic

Example in Product App:

```
public class Product {
   private Long id;
   private String name;
   private double price;
   private int quantity;
}
```

Model contains data and validation rules.

---

2) View

Represents:

- UI (HTML, JSP, Thymeleaf)

Example:
product-list.html

View displays data sent from Controller.

---

3) Controller

Handles:

- Incoming HTTP requests
- Calls service/business logic
- Sends data to view

Example:

```
@Controller
@RequestMapping("/products")
public class ProductController {
}
```

Real-Time Example (Product Management System)

User opens:

http://localhost:8080/products/list

Flow:

1. Controller receives request
2. Controller calls service
3. Service fetches product data
4. Controller adds data to Model
5. View renders product-list.html

---

2) Spring MVC Architecture

Spring MVC follows MVC pattern internally.

Core Components

1. DispatcherServlet
2. Handler Mapping
3. Controller
4. Model
5. View Resolver
6. View

Request Flow in Spring MVC

Step 1:
User sends request → Browser → Server

Step 2:
DispatcherServlet receives request
(It is the front controller)

Step 3:
HandlerMapping finds matching controller method

Step 4:
Controller method executes

Step 5:
Controller returns:

- View name
- Model data

Step 6:
ViewResolver resolves logical name to actual HTML file

Step 7:
View renders response to user

Example Flow in Your App

User hits:

/products/list

DispatcherServlet → ProductController → viewProducts()
Controller adds product list to model
Returns "product-list"
Thymeleaf renders product-list.html

---

3) Spring Web MVC Project with Annotations

Spring Boot simplifies configuration using annotations.

Important Annotations

@SpringBootApplication

Main class entry point

@SpringBootApplication

```
public class ProductAppApplication {
   public static void main(String[] args) {
      SpringApplication.run(ProductAppApplication.class, args);
   }
}
```

@Controller

Marks class as MVC controller.

```
@Controller
@RequestMapping("/products")
public class ProductController {
}
```

@RequestMapping

Defines base URL.

```
@RequestMapping("/products")
```

@GetMapping

Handles GET request.

```
@GetMapping("/list")
```

@PostMapping

Handles POST request.

```
@PostMapping("/save")
```

@PathVariable

Reads value from URL.

```
@GetMapping("/edit/{id}")
```

public String edit(@PathVariable Long id)

URL:
products/edit/5

@RequestParam

Reads value from query parameter.

URL:
products/search?name=Laptop

@GetMapping("/search")
public String search(@RequestParam String name)

@ModelAttribute

Binds form data to Java object.

@PostMapping("/save")
public String save(@ModelAttribute Product product)

@SessionAttributes

Stores model data in HTTP session.

@Valid

Triggers validation.

---

4) Spring MVC Data Binding using Model

What is Data Binding?

Data Binding means:

Automatically mapping:
Form fields → Java object properties

Example Form

```
<form th:action="@{/products/save}"
    th:object="${product}" method="post">

  <input type="text" th:field="*{name}" />
  <input type="number" th:field="*{price}" />
</form>
```

What Happens Internally?

User enters:

Name = Laptop
Price = 50000

Spring:

1. Reads form data
2. Creates Product object
3. Calls setName()
4. Calls setPrice()

All automatic.

This is Data Binding.

Using Model to Send Data to View

Controller:

```
@GetMapping("/list")
public String list(Model model) {
  model.addAttribute("products", service.getAllProducts());
```

```
    return "product-list";
}
```

Model is basically:

Map<String, Object>

Key = "products"
Value = List<Product>

Accessing Model Data in View

```
<tr th:each="p : ${products}">
    <td th:text="${p.name}"></td>
</tr>
```

${products} reads model attribute.

---

5) Reading Form Data in Spring MVC

There are 3 main ways.

---

1) Using @RequestParam

Used for simple values.

Example:

```
@PostMapping("/save")
public String save(
    @RequestParam String name,
    @RequestParam double price) {

    Product p = new Product();
    p.setName(name);
    p.setPrice(price);
```

```
}
```

Used when:
Few simple fields.

---

2) Using @ModelAttribute (Recommended)

Best for object binding.

```
@PostMapping("/save")
public String save(@ModelAttribute Product product) {
   service.saveProduct(product);
}
```

Automatically binds entire form.

Used when:
Form has many fields.

---

3) Using @RequestBody (REST APIs)

Used when:
Data comes in JSON format.

```
@PostMapping("/api/products")
public String add(@RequestBody Product product) {
}
```
